



Banker's Algorithm

- Multiple instances
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time





Data Structures for the Banker's Algorithm

- Let n = number of processes, and m = number of resources types.
- **Available**: Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max**: $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation**: $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need**: $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task
$$Need[i,j] = Max[i,j] - Allocation[i,j]$$





Safety Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize:

$Work = Available$

$Finish[i] = false$ for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) $Finish[i] = false$

(b) $Need_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2

4. If $Finish[i] == true$ for all i , then the system is in a safe state





Resource-Request Algorithm for Process P_i

Request = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	



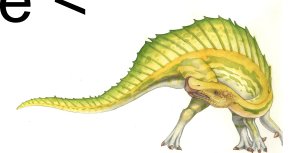


Example (Cont.)

- The content of the matrix *Need* is defined to be *Max – Allocation*

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria



$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i = 0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and Need₀ > Work

So P₀ must wait But Need ≤ Work

For $i = 1$ Step 2

Need₁ = 1, 2, 2

Finish [1] is false and Need₁ < Work

So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i = 2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and Need₂ > Work

So P₂ must wait

For $i = 3$ Step 2

Need₃ = 0, 1, 1

Finish [3] = false and Need₃ < Work

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i = 4$ Step 2

Need₄ = 4, 3, 1

Finish [4] = false and Need₄ < Work

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i = 0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and Need < Work

So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i = 2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and Need₂ < Work

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Banker's Algorithm in Operating System - GeeksforGeeks



Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?



Example: P_1 Request (1,0,2)

A B C
Request₁ = 1, 0, 2

To decide whether the request is granted we use Resource Request algorithm

$\begin{matrix} 1, 0, 2 & 1, 2, 2 \\ \text{Request}_1 < \text{Need}_1 \end{matrix}$
✓ Step 1

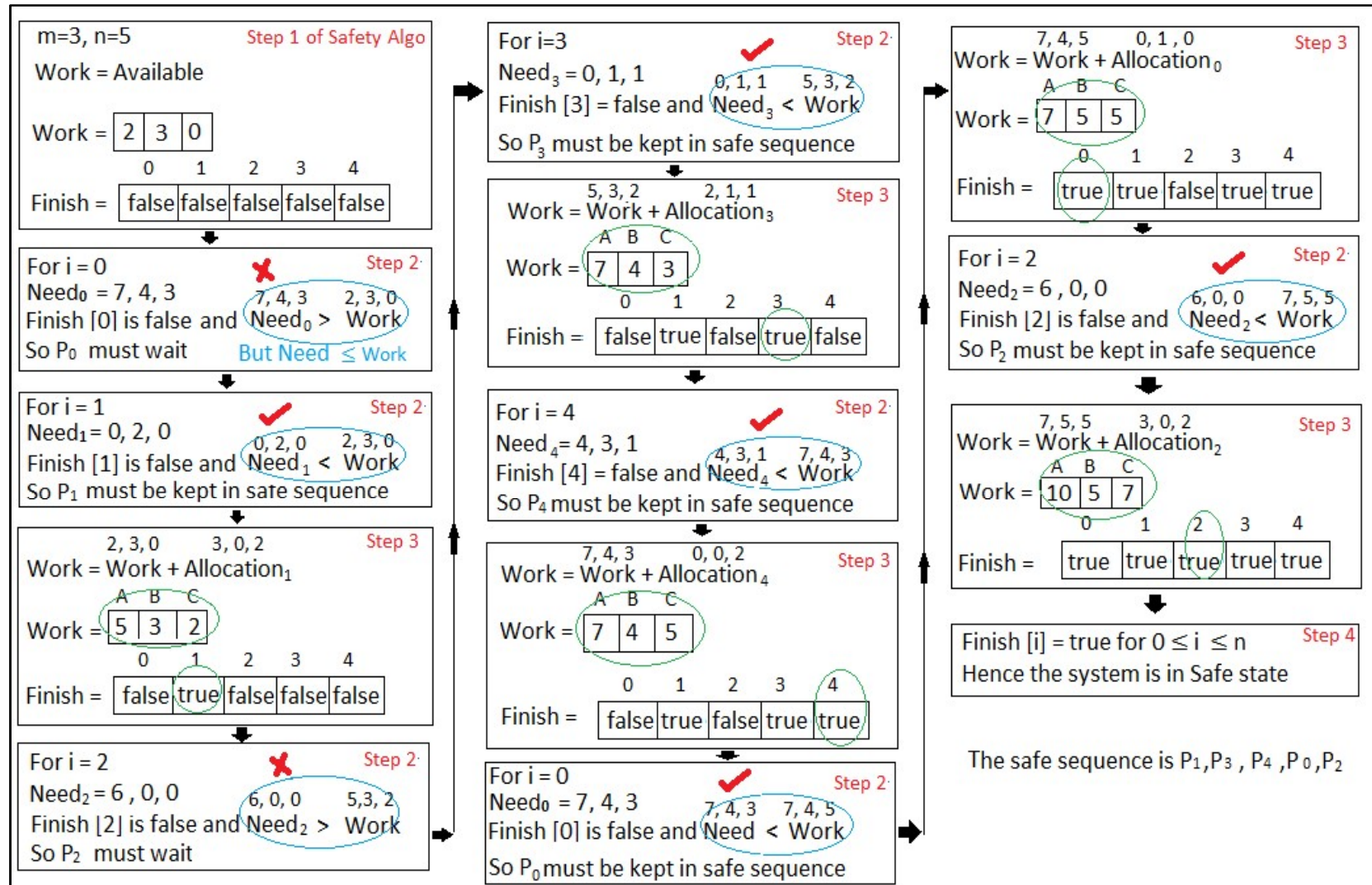
$\begin{matrix} 1, 0, 2 & 3, 3, 2 \\ \text{Request}_1 < \text{Available} \end{matrix}$
✓ Step 2

Step 3

Available = Available – Request₁
 Allocation₁ = Allocation₁ + Request₁
 Need₁ = Need₁ - Request₁

Process	Allocation	Need	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 4 3	2 3 0
P ₁	3 0 2	0 2 0	
P ₂	3 0 2	6 0 0	
P ₃	2 1 1	0 1 1	
P ₄	0 0 2	4 3 1	

Example: P_1 Request (1,0,2)



Hence the new system state is safe, so we can immediately grant the request for process P_1 .

Banker's Algorithm in Operating System - GeeksforGeeks